

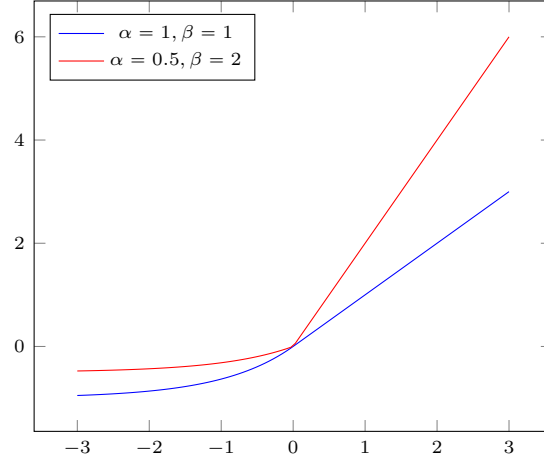


Figure 6: $\text{e1u}(\alpha, \beta x)$ where $\alpha \geq 0$ and $\beta \geq 0$ are hyper-parameters. The function $\text{e1u}(\alpha, \beta(q - \mathcal{G}_\theta(\mathcal{K})))$ with α low and β high is a soft constraint for penalty $(\mathcal{G}_\theta, q)$ suitable for learning.

is grounded using Tensorflow primitives such that LTN implements directly a Tensorflow graph. Due to Tensorflow built-in optimization, LTN is relatively efficient while providing the expressive power of first-order logic. Details on the implementation of the examples described in this section are reported in Appendix A. The implementation of the examples presented here is also available from the LTN repository on GitHub. Except when stated otherwise, the results reported are the average result over 10 runs using a 95% confidence interval. Every example uses a stable real product configuration to approximate the Real Logic operators and the Adam optimizer [35] with a learning rate of 0.001. Table A.3 in the Appendix gives an overview of the network architectures used to obtain the results reported in this section.

4.1. Binary Classification

The simplest machine learning task is binary classification. Suppose that one wants to learn a binary classifier A for a set of points in $[0, 1]^2$. Suppose that a set of positive and negative training examples is given. LTN uses the following language and grounding:

Domains:

points (denoting the examples).

Variables:

x_+ for the positive examples.

x_- for the negative examples.

x for all examples.

$\mathbf{D}(x) = \mathbf{D}(x_+) = \mathbf{D}(x_-) = \text{points}.$

Predicates:

$A(x)$ for the trainable classifier.

$\mathbf{D}_{\text{in}}(A) = \text{points}.$

Axioms:

$$\forall x_+ A(x_+) \quad (25)$$

$$\forall x_- \neg A(x_-) \quad (26)$$

Grounding:

$$\mathcal{G}(\text{points}) = [0, 1]^2.$$

$\mathcal{G}(x) \in [0, 1]^{m \times 2}$ ($\mathcal{G}(x)$ is a sequence of m points, that is, m examples).

$$\mathcal{G}(x_+) = \langle d \in \mathcal{G}(x) \mid \|d - (0.5, 0.5)\| < 0.09 \rangle.^{14}$$

$$\mathcal{G}(x_-) = \langle d \in \mathcal{G}(x) \mid \|d - (0.5, 0.5)\| \geq 0.09 \rangle.^{15}$$

$\mathcal{G}(A \mid \theta) : x \mapsto \text{sigmoid}(\text{MLP}_\theta(x))$, where MLP is a Multilayer Perceptron with a single output neuron, whose parameters θ are to be learned¹⁶.

Learning:

Let us define D the data set of all examples. The objective function with $\mathcal{K} = \{\forall x_+ A(x_+), \forall x_- \neg A(x_-)\}$ is given by $\text{argmax}_{\theta \in \Theta} \text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_{\theta, x \leftarrow D}(\phi)$.¹⁷ In practice, the optimizer uses the following loss function:

$$L = (1 - \text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_{\theta, x \leftarrow B}(\phi))$$

where B is a mini-batch sampled from D .¹⁸ The objective and loss functions depend on the following hyper-parameters:

- the choice of fuzzy logic operator semantics used to approximate each connective and quantifier,
- the choice of hyper-parameters underlying the operators, such as the value of the exponent p in any generalized mean,
- the choice of formula aggregator function.

Using the stable product configuration to approximate connectives and quantifiers, and $p = 2$ for every occurrence of A_{pME} , and using for the formula aggregator also A_{pME} with $p = 2$, yields the following satisfaction equation:

$$\begin{aligned} \text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_\theta(\phi) = & 1 - \frac{1}{2} \left(1 - \left(\frac{1}{|\mathcal{G}(x_+)|} \sum_{v \in \mathcal{G}(x_+)} (1 - \text{sigmoid}(\text{MLP}_\theta(v)))^2 \right)^{\frac{1}{2} \cdot 2} \right) \\ & + 1 - \left(1 - \left(\frac{1}{|\mathcal{G}(x_-)|} \sum_{v \in \mathcal{G}(x_-)} (\text{sigmoid}(\text{MLP}_\theta(v)))^2 \right)^{\frac{1}{2} \cdot 2} \right) \end{aligned}$$

¹⁴ $\mathcal{G}(x_+)$ are, by definition in this example, the training examples with Euclidean distance to the center $(0.5, 0.5)$ smaller than the threshold of 0.09.

¹⁵ $\mathcal{G}(x_-)$ are, by definition, the training examples with Euclidean distance to the centre $(0.5, 0.5)$ larger or equal to the threshold of 0.09.

¹⁶ $\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$

¹⁷The notation $\mathcal{G}_{x \leftarrow D}(\phi(x))$ means that the variable x is grounded with the data D (that is, $\mathcal{G}(x) := D$) when grounding $\phi(x)$.

¹⁸As usual in ML, while it is possible to compute the loss function and gradients over the entire data set, it is preferred to use mini-batches of the examples.

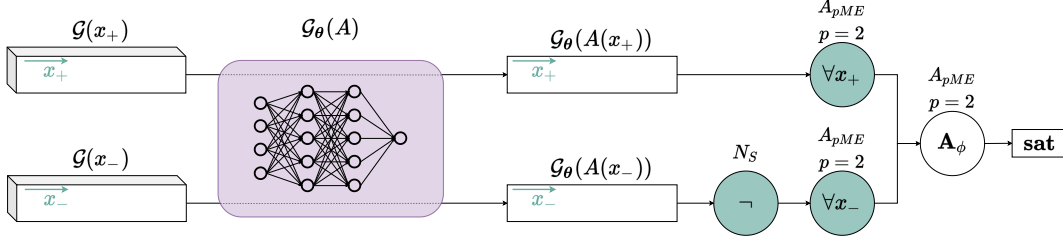


Figure 7: Symbolic Tensor Computational Graph for the Binary Classification Example. In the figure, $\mathcal{G}x_+$ and $\mathcal{G}x_-$ are inputs to the network $\mathcal{G}_\theta(A)$ and the dotted lines indicate the propagation of activation from each input through the network, which produces two outputs.

The computational graph of Figure 7 shows $\text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_\theta(\phi)$ as used with the above loss function.

We are therefore interested in learning the parameters θ of the MLP used to model the binary classifier. We sample 100 data points uniformly from $[0, 1]^2$ to populate the data set of positive and negative examples. The data set was split into 50 data points for training and 50 points for testing. The training was carried out for a fixed number of 1000 epochs using backpropagation with the Adam optimizer [35] with a batch size of 64 examples. Figure 8 shows the classification accuracy and satisfaction level of the LTN on both training and test sets averaged over 10 runs using a 95% confidence interval. The accuracy shown is the ratio of examples correctly classified, with an example deemed as being positive if the classifier outputs a value higher than 0.5.

Notice that a model can reach an accuracy of 100% while satisfaction of the knowledge base is yet not maximized. For example, if the threshold for an example to be deemed as positive is 0.7, all examples may be classified correctly with a confidence score of 0.7. In that case, while the accuracy is already maximized, the satisfaction of $\forall x_+ A(x_+)$ would still be 0.7, and can still improve until the confidence for every sample reaches 1.0.

This first example, although straightforward, illustrates step-by-step the process of using LTN in a simple setting. Notice that, according to the nomenclature of Section 3.3, measuring accuracy amounts to querying the *truth query* (respectively, the *generalization truth query*) $A(x)$ for all the examples of the training set (respectively, test set) and comparing the results with the classification threshold. In Figure 9, we show the results of such queries $A(x)$ after optimization. Next, we show how the LTN language can be used to solve progressively more complex problems by combining learning and reasoning.

4.2. Multi-Class Single-Label Classification

The natural extension of binary classification is a multi-class classification task. We first approach multi-class single-label classification, which assumes that each example is assigned to one and only one label.

For illustration purposes, we use the *Iris flower* data set [20], which consists of classification into three mutually exclusive classes; call these A , B , and C . While one could train three unary predicates $A(x)$, $B(x)$ and $C(x)$, it turns out to be more effective if this problem is modeled by a single binary predicate $P(x, l)$, where l is a variable denoting a multi-class label, in this case,